Centro de Investigacion y de Estudios Avanzados
del Instituto Politecnico Nacional
Unidad Guadalajara

Investigación y evaluación de modelos de inteligencia artificial aplicados al diseño RTL

Tipo de documento: Informe Tecnico de Pasantia de Investigacion

Pasante: Luis Carlos Montenegro Caballero

Asesor: Miguel Angel Rivera Acosta

Programa: Pasantia Academica 2026

Periodo: Enero – Marzo 2026

Guadalajara, Jalisco, Mexico

Marzo 2026

Contents

1 Introduccion **4**

1.1 Contexto y Motivacion 4

1.2 Objetivos del Benchmark 4

2 Metodologia **4**

2.1 Circuitos de Referencia 4

2.2 Niveles de Verificacion (L1–L4) 5

2.3 Estilos de Prompt (V1–V5) 6

2.4 Modelos Evaluados 7

2.5 Nota sobre Reproducibilidad: Correccion del Pipeline de RTLCoder . . 7

3 Resultados Globales — Los 11 Modelos **7**

3.1 Ranking General Consolidado 7

3.2 Analisis por Circuito — Los 11 Modelos 8

3.3 Hallazgo Central: Escala Inversa con Conocimiento Implicito 9

4 Analisis de Modelos Open-Source **10**

4.1 Ranking Ponderado 10

4.2 Pass@k por Circuito — Open-Source 10

4.3 Analisis Individual por Modelo 11

4.4 Taxonomia de Errores Open-Source 12

5 Analisis de Modelos Comerciales **12**

5.1 Condiciones de Evaluacion 12

5.2 Metricas Principales 13

5.3 Desempeno por Circuito 13

5.4 Analisis de Sintesis L4 — Celdas Logicas 14

5.5 Error Sistematico E-CAST-01 — Analisis Tecnico 14

6 Analisis Comparativo Global **15**

6.1 Open-Source vs. Comerciales: Resumen de Brechas 15

6.2 Hallazgos Comparativos Clave 16

7 Principios Derivados y Recomendaciones **16**

7.1 Tres Principios Metodologicos 16

7.2 Guia de Seleccion de Modelo por Tarea 17

8 Conclusiones **17**

A Fragmentos de Codigo RTL Representativos **19**

A.1 Error E-ARI-01: SUB Incorrecto en Qwen2.5-Coder-7B V1 19

A.2 Error E-FLAG-02: Overflow = Carry en Qwen2.5-Coder-7B V2 19

A.3 Error E-CAST-01: typedef enum sin Cast Explicito — Claude seq_det V1 20

A.4 Error E-SEM-01: Latch Inferido — ChatGPT alu V2 21

B Datos Crudos del Benchmark — Modelos Comerciales **22**

Abstract

Este informe presenta los resultados de un benchmark sistematico disenado para evaluar la capacidad de once modelos de lenguaje grande (Large Language Models, LLM) en la generacion de codigo RTL/HDL sintetizable en SystemVerilog. El estudio comprende dos etapas: la primera evalua ocho modelos open-source con parametros entre 1B y 7B, y la segunda incorpora los tres asistentes conversacionales comerciales de mayor adopcion industrial Gemini (Google), ChatGPT (OpenAI) y Claude (Anthropic). Todos los modelos fueron evaluados sobre tres circuitos de referencia (ALU de 8 bits, Adder Tree de cuatro entradas y Detector de Secuencia FSM "1011") mediante cuatro niveles de verificacion independientes: compilacion sintactica con `iverilog` (L1), correccion funcional por testbench (L2), verificacion formal exhaustiva con SymbiYosys/SVA (L3) y sintesis de area con Yosys (L4). El hallazgo central es que la barrera principal para la generacion RTL correcta no es sintactica sino semantica de dominio: ninguno de los once modelos supera Pass@k = 0 % en la ALU, cuya especificacion exige conocimiento implicito sobre flags aritmeticos en complemento a dos, mientras los modelos Tier-1 open-source alcanzan 66.7 % en verificacion formal (ventaja cualitativa que los tres modelos comerciales no logran igualar (L3 = 0 %)). Los modelos especializados RTLCoder-DeepSeek-v1.1 (88.5/100) y Qwen2.5-Coder-7B (83.9/100) lideran el ranking global, superando a Gemini (50.0), ChatGPT (49.0) y Claude (38.3).

Palabras clave: LLM, RTL, HDL, SystemVerilog, generacion de codigo, verificacion formal, benchmark, sintesis logica, FPGA.

1 Introduccion

1.1 Contexto y Motivacion

El diseno de hardware digital ha experimentado una transformacion significativa con la irrupcion de los modelos de lenguaje grande como herramientas de asistencia en la generacion de codigo HDL. Trabajos recientes como VerilogEval [1] y RTLBench [2] demuestran que los LLMs pueden compilar fragmentos de SystemVerilog con tasas superiores al 70 %, abriendo la posibilidad de integrarlos en flujos de diseno RTL asistidos por inteligencia artificial. Sin embargo, la tasa de compilacion es una metrica necesaria pero no suficiente: un diseno que compila puede sintetizar logica funcionalmente incorrecta, con flags aritmeticos erroneos o latches inferidos implicitamente, defectos que solo la verificacion formal puede detectar de forma exhaustiva.

Este trabajo presenta un benchmark multi-nivel que integra cuatro capas de evaluacion independientes y las aplica sobre once modelos organizados en dos grupos: ocho open-source y tres comerciales. El objetivo no es unicamente producir un ranking de rendimiento, sino identificar la naturaleza del cuello de botella que impide la generacion RTL correcta y derivar principios metodologicos aplicables al diseno de flujos de IA para hardware. La infraestructura experimental se ejecuto en un servidor Linux con GPU NVIDIA RTX 4090 (modelos open-source) y mediante APIs oficiales de cada proveedor (modelos comerciales), empleando OSS CAD Suite como entorno unificado de verificacion y sintesis.

1.2 Objetivos del Benchmark

Los objetivos especificos de esta investigacion son: (1) evaluar comparativamente once LLMs en la generacion de tres circuitos RTL de referencia bajo cinco estilos de prompt; (2) clasificar taxonomicamente los errores generados para identificar el tipo de conocimiento faltante en cada modelo; (3) comparar el desempeno de modelos open-source especializados versus asistentes comerciales de proposito general; y (4) derivar principios metodologicos para el uso responsable de LLMs en flujos de diseno RTL.

2 Metodologia

2.1 Circuitos de Referencia

Se seleccionaron tres circuitos que cubren niveles crecientes de complejidad estructural pero con variabilidad deliberada en la dependencia de conocimiento implicito de dominio. Esta seleccion permite separar la dificultad estructural de la dificultad

semantica, lo que constituye la hipotesis central del estudio.

Table 1: Circuitos de referencia del benchmark.

Circuito	**Tipo**	**Complejidad estructural**	**Conocimiento implicito**
ALU 8 bits	Combinacional	Baja	Alto (carry, overflow, borro
Adder Tree 4x8	Combinacional	Media	Bajo (semantica explicita)
Detector FSM "1011"	Secuencial	Alta	Medio (always_ff, solapami

La **ALU de 8 bits** implementa siete operaciones (ADD, SUB, AND, OR, XOR, SHL, SHR) con cuatro flags de estado: `Zero`, `Carry`, `Overflow` y `Negative`. La generacion correcta del flag `Overflow` en complemento a dos requiere que el modelo haya internalizado la relacion algebraica:

$$\text{Overflow} = A[7] \oplus B[7] \oplus \text{Result}[7] \oplus \text{CarryIn} \tag{1}$$

Esta relacion no es derivable del lenguaje natural sin conocimiento previo de la aritmetica con signo, lo que convierte a la ALU en el circuito de mayor exigencia semantica del benchmark.

El **Adder Tree** recibe cuatro operandos de 8 bits y produce su suma en 10 bits mediante una estructura jerarquica de dos niveles. La salida de 10 bits es necesaria para evitar desbordamiento ($255 \times 4 = 1020 < 2^{10}$), y su especificacion puede derivarse directamente de la logica del circuito sin conocimiento implicito adicional. El **Detector de Secuencia FSM** implementa un reconocedor del patron binario `1011` con solapamiento, operando como maquina de Moore con cuatro estados (S0–S3) y reset activo-bajo.

2.2 Niveles de Verificacion (L1–L4)

El pipeline implementa cuatro niveles de evaluacion independientes y jerarquicos:

L1 — Compilacion sintactica: Verificacion con `iverilog -g2012`. Un archivo supera L1 si y solo si la compilacion concluye sin errores. L1 detecta errores de sintaxis, declaraciones de puertos incorrectas y referencias a tipos no definidos, pero no detecta errores logicos silenciosos.

L2 — Correccion funcional: Ejecucion de testbench con 20 vectores de prueba para Adder Tree y 41 vectores para la ALU. Se registra el numero de casos que pasan (`L2_pass_count`) y la fraccion de exito (`L2_functional`). El testbench cubre casos extremos: $A = 0$, $A = 255$, overflow positivo, overflow negativo y carry en operaciones aritmeticas.

L3 — Verificacion formal: Verificacion exhaustiva con SymbiYosys y propiedades SVA mediante Bounded Model Checking a profundidad 20. L3 garantiza correccion para todos los posibles vectores de entrada, a diferencia del testbench que cubre un subconjunto finito.

L4 — Sintesis de area: Sintesis con Yosys reportando numero de celdas logicas como proxy de area de silicio. Un latch inferido implicitamente por cobertura incompleta de ramas en `always_comb` constituye un fallo de L4 aunque el codigo compile y pase el testbench.

La puntuacion final por modelo se calcula como:

$$S = 0{,}30 \cdot C_{L1} + 0{,}40 \cdot C_{L2} + 0{,}20 \cdot C_{L3} + 0{,}10 \cdot (1 - H) \tag{2}$$

donde C_{L1}, C_{L2}, C_{L3} son las tasas de exito en compilacion, funcion y verificacion formal respectivamente, y H es la tasa de alucinacion HDL.

2.3 Estilos de Prompt (V1–V5)

Cada circuito fue evaluado con cinco variantes de prompt que cubren el espectro desde la especificacion formal hasta el lenguaje conversacional. **V1 (Especificacion tecnica formal)** describe la interfaz, las operaciones y los requerimientos de sintesis con terminologia precisa. **V2 (Lenguaje natural conversacional)** emplea descripcion funcional informal sin terminologia RTL especifica, representando el caso de un usuario sin experiencia en HDL. **V3 (Tabla de comportamiento)** codifica la funcion directamente mediante tabla de verdad o tabla de transiciones, externalizando el conocimiento de dominio en una representacion estructurada. **V4 (Bloques funcionales RTL)** proporciona un esqueleto de codigo con declaracion de modulo y comentarios `TODO`, restringiendo el espacio de solucion del modelo. **V5 (Guia paso a paso)** induce razonamiento explicito previo a la generacion, mimificando la tecnica de chain-of-thought prompting.

2.4 Modelos Evaluados

Table 2: Modelos evaluados en el benchmark. OS = Open-Source, COM = Comercial.

Modelo	Tipo	Parametros	Ejecucion
RTLCoder-DeepSeek-v1.1	OS	6.7B	Servidor RTX 4090, FP16
Qwen2.5-Coder-7B	OS	7B	Servidor RTX 4090, FP16
RTLCoder (corregido)	OS	6.7B	Google Colab + OSS CAD Suite
StarCoder2-7B	OS	7B	Servidor RTX 4090, FP16
DeepSeek-Coder-V2-Lite	OS	2.4B	Servidor RTX 4090, FP16
TinyLlama-1.1B	OS	1.1B	Servidor RTX 4090, FP16
DeepSeek-R1-1.5B	OS	1.5B	Servidor RTX 4090, FP16
Llama-3.2-1B	OS	1B	Servidor RTX 4090, FP16
Gemini	COM	n/d	API oficial Google
ChatGPT	COM	n/d	API oficial OpenAI
Claude	COM	n/d	API oficial Anthropic

2.5 Nota sobre Reproducibilidad: Correccion del Pipeline de RTLCoder

Durante la fase inicial de evaluacion se detecto un artefacto en el pipeline de generacion que asigno incorrectamente una puntuacion de 6.6/100 a RTLCoder (version original). El error consistio en una condicion de race condition en el script de orquestacion que impidio que los archivos generados por este modelo fueran correctamente evaluados por los niveles L2–L4. Detectado el problema, se realizo una re-evaluacion completa del modelo en Google Colab utilizando la misma version de OSS CAD Suite, obteniendo la puntuacion corregida de 74.3/100, que lo ubica en el tercer lugar del ranking open-source. Todos los resultados reportados en este documento corresponden a los valores corregidos.

3 Resultados Globales — Los 11 Modelos

3.1 Ranking General Consolidado

La Tabla 3 presenta el ranking final de los once modelos evaluados. El resultado mas prominente es que los cinco modelos open-source de mayor desempeno ocupan las primeras cinco posiciones del ranking, con los tres modelos comerciales ubicandose en los puestos 6 a 8. Este resultado es contraintuitivo en el contexto de la narrativa dominante sobre la superioridad de los modelos fundacionales de gran escala, y revela

que la especializacion de dominio en RTL confiere una ventaja sistematica que la escala de proposito general no compensa, al menos en el contexto de verificacion formal.

Table 3: Ranking general consolidado — 11 modelos. Puntuacion segun ecuacion (2).

#	**Modelo**	**Tipo**	**Comp. %**	**Func. %**	**Formal %**	**Score**
1	RTLCoder-DeepSeek-v1.1	OS	100.0	84.0	66.7	88.5
2	Qwen2.5-Coder-7B	OS	73.3	90.7	66.7	83.9
3	RTLCoder (corr. †)	OS	53.3	93.1	60.0	74.3
4	StarCoder2-7B	OS	53.3	84.5	60.0	72.7
5	DeepSeek-Coder-V2-Lite	OS	53.3	53.5	26.7	56.7
6	Gemini	COM	66.7	66.7	0.0	50.0
7	ChatGPT	COM	66.7	66.7	0.0	49.0
8	Claude	COM	66.7	33.3	0.0	38.3
9	TinyLlama-1.1B	OS	0.0	0.0	0.0	13.0
10	DeepSeek-R1-1.5B	OS	0.0	0.0	0.0	10.8
11	Llama-3.2-1B	OS	0.0	0.0	0.0	9.0
† Resultado original incorrecto: 6.6/100. Ver Seccion 2.5.						

El ranking revela tres agrupaciones naturales. El **Tier-1** (Score >70) esta integrado por RTLCoder-DS-v1.1, Qwen2.5-Coder-7B, RTLCoder corregido y StarCoder2-7B: modelos con capacidad real de compilacion y verificacion formal parcial. El **Tier-2** (Score 35–70) incluye DeepSeek-Coder-V2-Lite y los tres modelos comerciales, que muestran compilacion y funcionalidad razonable pero sin capacidad de verificacion formal. El **Tier-3** (Score <20) agrupa a los tres modelos de menor escala open-source, que no logran compilar en ninguno de los tres circuitos.

3.2 Analisis por Circuito — Los 11 Modelos

La Tabla 4 presenta la tasa de correctitud funcional Pass@5 por circuito para todos los modelos. El hallazgo mas relevante es la universalidad del fallo en la ALU: los once modelos evaluados, sin excepcion, obtienen Pass@k = 0 % en el circuito aritmetico de 8 bits. Este resultado es notable porque la ALU es el circuito mas simple en terminos de lineas de codigo y complejidad estructural, pero el mas exigente en terminos de conocimiento de dominio implicito.

Table 4: Pass@5 funcional (L2) por circuito — los 11 modelos.

Modelo	**Tipo**	**ALU**	**Adder Tree**	**FSM seq_det**
RTLCoder-DeepSeek-v1.1	OS	0 %	100 %	100 %
Qwen2.5-Coder-7B	OS	0 %	100 %	100 %
RTLCoder (corr.)	OS	0 %	100 %	0 %
StarCoder2-7B	OS	0 %	100 %	0 %
DeepSeek-Coder-V2-Lite	OS	0 %	100 %	0 %
Gemini	COM	100 %	100 %	0 % (L1 fail)
ChatGPT	COM	100 %	100 %	0 % (L1 fail)
Claude	COM	0 %	100 %	0 % (L1 fail)
TinyLlama-1.1B	OS	0 %	0 %	0 %
DeepSeek-R1-1.5B	OS	0 %	0 %	0 %
Llama-3.2-1B	OS	0 %	0 %	0 %

El Adder Tree es el circuito con mayor tasa de exito: siete de los once modelos lo resuelven correctamente al 100 %. Los tres modelos Tier-3 open-source fallan incluso en este circuito por incapacidad de compilacion. La FSM presenta un patron diferenciado: solo los dos modelos Tier-1 mas especializados (RTLCoder-DS y Qwen2.5) la resuelven entre los open-source, mientras los tres modelos comerciales fallan en L1 por una incompatibilidad de dialecto SystemVerilog que se analiza en detalle en la Seccion 5.5.

3.3 Hallazgo Central: Escala Inversa con Conocimiento Implicito

El patron observado en la Tabla 4 confirma la hipotesis central del estudio: la calidad de generacion RTL escala inversamente con la dependencia de conocimiento de dominio implicito en la especificacion del circuito. La ALU, cuya correctitud requiere internalizar la semantica de la aritmetica en complemento a dos (incluyendo la distincion entre carry-out como desbordamiento sin signo y overflow como desbordamiento con signo), es generalmente insoluble para todos los modelos evaluados. El Adder Tree, cuya especificacion determina explicitamente la estructura jerarquica y el ancho de bus requerido, es solucionado correctamente por todos los modelos con capacidad de compilacion suficiente. La FSM ocupa una posicion intermedia: su semantica de solapamiento no es inmediatamente obvia del lenguaje natural, pero puede codificarse en una tabla de transiciones que la hace accesible para modelos suficientemente especializados.

4 Analisis de Modelos Open-Source

4.1 Ranking Ponderado

La Tabla 5 detalla el ranking de los ocho modelos open-source con sus metricas desagregadas. La columna de alucinaciones HDL (H) registra la frecuencia con que el modelo genera construcciones inexistentes o invalidas en Verilog/SystemVerilog, detectadas mediante analisis estatico del codigo generado.

Table 5: Ranking ponderado — modelos open-source. H = tasa de alucinacion HDL.

#	**Modelo**	**Comp. %**	**Func. %**	**Formal %**	**H %**	**Score**
1	RTLCoder-DeepSeek-v1.1	100.0	84.0	66.7	0.0	88.5
2	Qwen2.5-Coder-7B	73.3	90.7	66.7	0.0	83.9
3	RTLCoder (corr.)	53.3	93.1	60.0	26.7	74.3
4	StarCoder2-7B	53.3	84.5	60.0	20.0	72.7
5	DeepSeek-Coder-V2-Lite	53.3	53.5	26.7	20.0	56.7
6	TinyLlama-1.1B	0.0	0.0	0.0	71.4	13.0
7	DeepSeek-R1-1.5B	0.0	0.0	0.0	93.3	10.8
8	Llama-3.2-1B	0.0	0.0	0.0	100.0	9.0

4.2 Pass@k por Circuito — Open-Source

Table 6: Pass@1 y Pass@5 por circuito — modelos open-source.

Modelo	**ALU**		**Adder Tree**		**FSM**	
	@1	**@5**	**@1**	**@5**	**@1**	**@5**
RTLCoder-DeepSeek-v1.1	0 %	0 %	100 %	100 %	100 %	100 %
Qwen2.5-Coder-7B	0 %	0 %	100 %	100 %	100 %	100 %
RTLCoder (corr.)	0 %	0 %	100 %	100 %	0 %	0 %
StarCoder2-7B	0 %	0 %	100 %	100 %	0 %	0 %
DeepSeek-Coder-V2-Lite	0 %	0 %	100 %	100 %	0 %	0 %
TinyLlama-1.1B	0 %	0 %	0 %	0 %	0 %	0 %
DeepSeek-R1-1.5B	0 %	0 %	0 %	0 %	0 %	0 %
Llama-3.2-1B	0 %	0 %	0 %	0 %	0 %	0 %

La igualdad entre Pass@1 y Pass@5 en RTLCoder-DS y Qwen2.5 para los circuitos Adder Tree y FSM indica una consistencia perfecta: cualquier variante de prompt

produce el mismo resultado, lo que refleja que el conocimiento del dominio esta consolidado en el modelo y no depende de la formulacion del prompt. Esta consistencia es una propiedad deseable en un entorno de produccion, donde no puede garantizarse que el usuario formule el prompt optimo.

4.3 Analisis Individual por Modelo

RTLCoder-DeepSeek-v1.1 obtiene la puntuacion global mas alta (88.5/100) y es el unico modelo del benchmark que alcanza 100 % en compilacion. Su arquitectura base es DeepSeek-Coder con fine-tuning sobre un corpus curado de aproximadamente 100K ejemplos de codigo RTL sintetizable, lo que le confiere un vocabulario semantico ajustado al dominio HDL. Su tasa de alucinaciones del 0 % confirma que el fine-tuning elimina efectivamente la generacion de construcciones invalidas. La unica dimension en que no lidera es la funcional pura (84.0 % vs. 93.1 % de RTLCoder corregido), diferencia atribuible a variaciones en las versiones del corpus de entrenamiento.

Qwen2.5-Coder-7B demuestra que un modelo de codigo general de suficiente escala y calidad puede aproximarse al desempeno de modelos RTL-especializados. Con 83.9/100 y 0 % de alucinaciones, el modelo aprovecha un preentrenamiento de alta calidad en codigo de multiples lenguajes para generalizar correctamente hacia Verilog/SystemVerilog. Su tasa funcional de 90.7 % supera a RTLCoder-DS en esa dimension, aunque su tasa de compilacion (73.3 %) es inferior, lo que sugiere mayor variabilidad en la correctitud sintactica.

RTLCoder (corregido) y **StarCoder2-7B** comparten identicas tasas de compilacion (53.3 %) y verificacion formal (60.0 %), pero difieren en su comportamiento con la FSM: ninguno de los dos resuelve el detector de secuencia, a diferencia de los dos lideres del ranking. Esta diferencia puede atribuirse a la profundidad del corpus de entrenamiento en diseno secuencial sintetizable con construcciones `always_ff`.

DeepSeek-Coder-V2-Lite representa el Tier-2 open-source. Con 2.4B parametros activos en arquitectura Mixture-of-Experts, compite razonablemente con modelos de mayor parametrizacion nominal, pero su tasa funcional de 53.5 % y formal de 26.7 % lo situan claramente por debajo del Tier-1. La presencia de alucinaciones (20.0 %) indica que la escala reducida afecta la coherencia semantica del codigo generado.

Los tres modelos **Tier-3** (TinyLlama-1.1B, DeepSeek-R1-1.5B, Llama-3.2-1B) no logran compilar en ninguno de los tres circuitos, con tasas de alucinacion entre 71.4 % y 100 %. Estos modelos son inadecuados para generacion RTL sin supervision experta y no son considerados para ninguna recomendacion de uso practico.

4.4 Taxonomia de Errores Open-Source

El analisis del codigo generado por los modelos open-source permite identificar cinco categorias principales de error, documentadas en la Tabla 7.

Table 7: Taxonomia de errores en generacion RTL — modelos open-source.

Codigo	Categoria	Descripcion	Modelo(s)
E-ARI-01	Aritmético / Op. errónea	SUB usa suma precalculada $(a + b) - 1$	Qwen2.5 V1
E-ARI-02	Aritmético / Desbord.	`result[8]` sobre señal 8 bits	StarCoder2 V2/V3
E-FLAG-01	Flags / Carry no aislado	carry asignado fuera del `case`	Qwen2.5 V1
E-FLAG-02	Flags / Overflow semantico	`overflow = carry` para ADD	Qwen2.5 V2
E-SEM-01	Semantica RTL / Latch	Señal no asignada en todos los branches	DeepSeek-V2L
E-IFACE-01	Interfaz / Puerto ancho	`op` declarado `[7:0]` en vez de `[3:0]`	StarCoder2 V2/V3
E-HALLUC-01	Alucinacion / Repeticion	Modulo declarado 15+ veces sin cuerpo	StarCoder2 V4/V5
E-HALLUC-02	Alucinacion / No sint.	`$display`, `#delays` en RTL sintetizable	DeepSeek, RTLCo

Los errores de categoria E-FLAG son los mas significativos desde la perspectiva del diseno digital porque producen circuitos que compilan, pasan parcialmente el testbench y fallan solo ante vectores de prueba especificos que cubran los casos limite de carry y overflow. Esta clase de error es especialmente peligrosa en entornos sin verificacion formal, donde un testbench incompleto puede certificar incorrectamente un circuito defectuoso.

5 Analisis de Modelos Comerciales

5.1 Condiciones de Evaluacion

Los tres modelos comerciales (Gemini (Google), ChatGPT (OpenAI) y Claude (Anthropic)) fueron evaluados sobre los mismos tres circuitos y cinco variantes de prompt que los modelos open-source, resultando en 15 casos canónicos por modelo (45 en total). Cada caso fue ejecutado dos veces (dos archivos independientes por variante) para verificar consistencia, produciendo 90 registros en el dataset. El pipeline de evaluacion L1–L4 es identico al empleado para los modelos open-source, garantizando la comparabilidad metodologica de los resultados.

5.2 Metricas Principales

Table 8: Metricas principales — modelos comerciales.

Modelo	L1 Comp. %	L2 Func. %	L3 Formal %	L4 Sintesis	Score
Gemini	66.7	66.7	0.0	Parcial	50.0
ChatGPT	66.7	66.7	0.0	Parcial	49.0
Claude	66.7	33.3	0.0	Parcial	38.3

Los tres modelos comparten una tasa de compilacion identica del 66.7 %, lo que corresponde exactamente a los circuitos ALU y Adder Tree (10 archivos de 15 posibles por modelo). El circuito seq_det falla en L1 para los tres modelos por el error E-CAST-01 analizado en la Seccion 5.5. La verificacion formal L3 = 0 % es identica para los tres, aunque no implica que el codigo sea logicamente incorrecto: el verificador formal requiere propiedades SVA especificas cuya satisfaccion depende tambien de la completitud de la especificacion.

5.3 Desempeno por Circuito

Table 9: Pass@5 funcional (L2) por circuito — modelos comerciales.

Modelo	ALU	Adder Tree	Seq. Det. (FSM)
Gemini	100 %	100 %	0 % (L1 fail)
ChatGPT	100 %	100 %	0 % (L1 fail)
Claude	0 %	100 %	0 % (L1 fail)

Gemini y ChatGPT alcanzan 100 % en la ALU ,resultado que ninguno de los ocho modelos open-source logra , lo que constituye el hallazgo comparativo mas relevante del benchmark. Claude, en contraste, obtiene 0 % en la ALU, alineandose con todos los modelos open-source en este circuito. El Adder Tree es resuelto correctamente por los tres modelos comerciales en todas las variantes de prompt.

5.4 Analisis de Sintesis L4 — Celdas Logicas

Table 10: Celdas logicas (Yosys) por circuito — modelos comerciales. Valores exactos del dataset experimental.

Modelo	Variante	ALU (celdas)	Adder Tree (celdas)
Claude	v1, v2, v3, v5	344	137
Claude	v4	361	137
ChatGPT	v1	395	137
ChatGPT	v2	0 (latch – L4 fail)	137
ChatGPT	v3	399	137
ChatGPT	v4	424	137
ChatGPT	v5	395	137
Gemini	v1, v2, v3, v5	395	137
Gemini	v4	400	137

La uniformidad del Adder Tree (137 celdas en los tres modelos para todas las variantes) confirma que los tres implementan el mismo circuito correcto de manera consistente. En la ALU, la diferencia entre Claude (≈344–361 celdas) y ChatGPT/Gemini (≈395–424 celdas) es estadisticamente significativa: una reduccion de ∼51 celdas (−13 %) respecto al rango ChatGPT/Gemini. Dado que el testbench utilizado en el pipeline incluye operaciones adicionales (SLT, MUL, NOP) no cubiertas por la especificacion basica de 7 operaciones, el conteo de celdas reducido de Claude es evidencia directa de que su implementacion no cubre el set completo de operaciones exigido por el testbench, lo que explica causalmente el Pass@k = 0 % en L2.

El caso de ChatGPT v2 (ALU) merece atencion especial: el codigo compila (L1=PASS) y pasa el testbench funcional (L2=PASS), pero falla en sintesis (L4=FAIL) por inferencia de latch(error E-SEM-01). El mensaje de Yosys registrado en el dataset es: `ERROR: Latch inferred for signal \alu.\tmp from always_comb process`. Este error indica que la senal `tmp` no es asignada en todos los posibles branches del bloque `always_comb`, lo que en hardware real produce un latch no intencional con comportamiento dependiente del estado previo.

5.5 Error Sistematico E-CAST-01 — Analisis Tecnico

El fallo de compilacion de los tres modelos comerciales en el circuito seq_det es sistematico y uniforme: todas las variantes V1–V5 de los tres modelos fallan con exactamente 5 errores del tipo `"This assignment requires an explicit cast"` en las lineas donde se asignan estados a la variable de estado enumerada.

Los tres modelos generan codigo FSM empleando `typedef enum logic [1:0] {S0, S1, S2, S3} state_t`, que es un patron valido en SystemVerilog-2012. Sin embargo, `iverilog` compilado con `-g2012` requiere que las asignaciones entre tipos enumerados usen un cast explicito de la forma `state_t'(valor)`. Este requisito esta presente en simuladores comerciales (ModelSim, VCS, Xcelium) pero su manejo difiere entre herramientas. El codigo generado por los modelos comerciales seria valido y compilaria sin errores en los simuladores de referencia de la industria, pero falla en `iverilog` bajo las condiciones del pipeline.

La implicacion metodologica es doble. Primero, el pipeline es uniformemente justo: el mismo compilador, los mismos flags y las mismas condiciones se aplican a todos los modelos, por lo que reportar L1=FAIL para estos casos es correcto dentro del marco del benchmark. Segundo, el hallazgo indica que el criterio de seleccion del compilador de referencia es una variable de control que debe documentarse explicitamente en cualquier benchmark de generacion RTL, dado que puede determinar el resultado de circuitos correctamente implementados.

6 Analisis Comparativo Global

6.1 Open-Source vs. Comerciales: Resumen de Brechas

El analisis comparativo entre los grupos open-source y comercial revela cuatro brechas estructurales que caracterizan las fortalezas y limitaciones de cada paradigma de modelo.

Brecha 1 — Verificacion Formal: Los tres modelos comerciales obtienen L3 = 0.0 %, equivalente al Tier-3 open-source. Los modelos Tier-1 open-source (RTLCoder-DS, Qwen2.5) alcanzan 66.7 % en verificacion formal. Esta brecha es la mas significativa desde la perspectiva del diseno digital profesional, donde la verificacion formal es el estandar de oro para circuitos criticos. La ventaja de los modelos open-source especializados en esta dimension es cualitativa, no solo cuantitativa: son los unicos modelos capaces de generar codigo que satisface propiedades formales verificables matematicamente.

Brecha 2 — ALU Funcional: Gemini y ChatGPT alcanzan 100 % en la ALU (L2), resultado que no logra ningun modelo open-source. Claude coincide con los open-source en 0 %. Este hallazgo es contraintuitivo: los modelos de proposito general de gran escala superan a los especializados en RTL en el circuito mas semanticamente exigente. Una hipotesis plausible es que la mayor escala de preentrenamiento de los modelos comerciales les ha permitido internalizar la semantica de la aritmetica en complemento a dos desde fuentes no-RTL (libros de texto, documentacion de arquitecturas

de procesadores, codigo C/C++ de operaciones bit a bit), mientras que los modelos open-source mas pequenos dependen de que esta semantica este presente explicitamente en el corpus RTL de fine-tuning.

Brecha 3 — Alucinaciones: Los tres modelos comerciales presentan 0 % de alucinaciones HDL, equivalente a los modelos Tier-1 open-source. Los modelos Tier-3 open-source exhiben tasas entre 71.4 % y 100 %, lo que los hace inviables para uso en flujos de sintesis. La eliminacion de alucinaciones parece requerir una escala de preentrenamiento o fine-tuning que los modelos de menos de 2B parametros no alcanzan.

Brecha 4 — Coherencia de Sintetizabilidad: Los tres modelos comerciales generan codigo sintetizable consistente cuando compilan (L4 = PASS en la mayoria de variantes). Los modelos open-source Tier-2 y Tier-3 mezclan con frecuencia construcciones de simulacion (`$display`, retardos `#N`) con codigo RTL, produciendo disenos no sintetizables que no son detectados hasta el nivel L4.

6.2 Hallazgos Comparativos Clave

El hallazgo central del benchmark, que la capacidad de generacion RTL correcta escala inversamente con la dependencia de conocimiento de dominio implicito, se cumple de manera general e independiente del grupo al que pertenece el modelo. Tanto los especializados como los comerciales fallan en la ALU; tanto los especializados Tier-1 como los comerciales superan el Adder Tree.

La ventaja decisiva de los modelos open-source especializados (RTLCoder-DS, Qwen2.5) sobre los comerciales reside exclusivamente en la verificacion formal: son los unicos que producen codigo cuya correctitud puede ser demostrada matematicamente mediante model checking, lo que los hace los unicos aptos para flujos que requieren garantias de correccion exhaustiva. Los modelos comerciales son competitivos en tareas sin requisito formal, pero no pueden ofrecer las garantias matematicas que el diseno de hardware critico demanda.

7 Principios Derivados y Recomendaciones

7.1 Tres Principios Metodologicos

P1 — El cuello de botella es semantico, no sintactico. Los LLMs han aprendido la gramatica de SystemVerilog con suficiente fidelidad para generar codigo compilable. La barrera es la fisica implicita del hardware: convenciones de representacion numerica, propagacion de flags aritmeticos y semantica de sintesis versus simulacion. El foco de mejora debe ser el corpus de preentrenamiento en HDL sintetizable con semantica correcta de flags, no la arquitectura del modelo.

P2 — El formato de la especificacion es una variable de ingenieria. La diferencia entre una especificacion narrativa (V2) y una tabla de comportamiento (V3) es medible en tasa funcional, especialmente para modelos bajo 10B parametros. Una especificacion tabular explicita actua como prompt engineering orientada a hardware y puede compensar parcialmente el deficit de conocimiento de dominio. El diseno del prompt debe tratarse como una etapa formal del flujo de diseno, no como una actividad ad hoc.

P3 — La verificacion formal no es opcional. Un testbench puede aprobar logica de flags incorrecta si los vectores de prueba no cubren los casos limite de carry y overflow. La verificacion formal con BMC a profundidad 20 sobre propiedades SVA es el filtro indispensable antes de sintesis cuando el codigo proviene de un LLM. El nivel L3 del pipeline debe considerarse obligatorio en cualquier flujo de diseno digital asistido por IA.

7.2 Guia de Seleccion de Modelo por Tarea

Table 11: Recomendaciones de uso por tarea y grupo de modelo.

Tarea RTL	**Modelo recomendado**	**Observacion**
Prototipado rapido (adder, mux, decoder)	Cualquier Tier-1 o COM	L2 confiable, bajo riesgo
ALU / aritmetica con flags	Gemini o ChatGPT	L2=100 %; L3=0 % (requiere FV externa)
FSM secuencial con `iverilog`	RTLCoder-DS, Qwen2.5	COM fallan por E-CAST-01
Sintesis con garantia formal	RTLCoder-DS, Qwen2.5	Unicos con L3 > 0
Iteracion conversacional sin requisito formal	Gemini, ChatGPT	0 % alucinaciones, codigo limpio
Educacion / prototipado ilustrativo	Cualquier Tier-1	Comentarios explicativos utiles

8 Conclusiones

Este trabajo comenzó como un ejercicio de evaluación comparativa y derivó en una investigación sobre los límites cognitivos fundamentales de los modelos de lenguaje grande en el dominio del hardware digital. Los resultados no solo establecen un ranking de desempeño, sino que revelan la naturaleza estructural de las barreras que impiden

la generación RTL confiable, independientemente del paradigma de entrenamiento del modelo. Las conclusiones principales son las siguientes.

Primero, los modelos especializados open-source superan a los comerciales en contextos RTL con requisito de verificación formal. RTLCoder-DeepSeek-v1.1 (88.5/100) y Qwen2.5-Coder-7B (83.9/100) lideran el ranking global, superando a Gemini (50.0), ChatGPT (49.0) y Claude (38.3). La ventaja no es marginal: los modelos Tier-1 open-source alcanzan verificación formal parcial del 66.7 %, frente al 0 % de los tres modelos comerciales. En entornos donde la corrección matemáticamente demostrable es un requisito, los modelos especializados son los únicos candidatos viables.

Segundo, el límite semántico de flags aritméticos es general e independiente de la escala del modelo. Ninguno de los once modelos evaluados supera Pass@k = 0 % en la ALU de 8 bits, incluyendo aquellos con preentrenamiento específico en RTL y los asistentes comerciales de mayor adopción industrial. Este resultado confirma que la barrera no es sintáctica ni arquitectónica, sino semántica: la correcta internalizacion de la aritmética en complemento a dos, incluyendo la distinción entre carry-out como desbordamiento sin signo y overflow como desbordamiento con signo, está ausente en el corpus de preentrenamiento de todos los modelos evaluados. Resolver este límite requiere intervención deliberada en los datos de entrenamiento, no solo en la escala del modelo.

Tercero, los modelos comerciales son competitivos en tareas RTL sin requisito de verificación formal. Gemini y ChatGPT igualan a los modelos Tier-1 en el Adder Tree y los superan en la ALU funcional (100 % frente a 0 % en L2), con 0 % de alucinaciones HDL y código sintetizable consistente en todas las variantes evaluadas. Esta competitividad los hace adecuados para flujos de prototipado rápido y asistencia conversacional, siempre que el diseño resultante sea sometido a verificación formal mediante herramientas externas antes de síntesis.

Cuarto, la verificación formal es la frontera de diferenciación real entre grupos de modelos. La métrica L3 es la única que separa cualitativamente a los modelos Tier-1 open-source del resto del campo evaluado. Mientras que las diferencias en compilación y funcionalidad son graduales, la brecha en verificación formal es binaria: o el modelo genera código que satisface propiedades SVA demostrables, o no lo hace. En flujos de diseño que exigen garantías matemáticas de corrección (circuitos aritméticos, controladores de seguridad, lógica de arbitraje) los modelos comerciales deben complementarse obligatoriamente con herramientas de verificación formal externas, sin excepción.

Quinto, la metodología es la contribución más duradera de este trabajo.

El benchmark multi-nivel L1–L4, la taxonomía de errores estructurada en seis categorías (E-ARI, E-FLAG, E-SEM, E-IFACE, E-HALLUC, E-CAST) y el protocolo de evaluación estandarizado con cinco variantes de prompt por circuito constituyen un marco reproducible y extensible. A medida que los modelos de lenguaje evolucionen, en escala, especialización y técnicas de alineación , este framework permitirá comparaciones longitudinales rigurosas que aíslen el progreso real del ruido estadístico. La próxima extensión natural es la evaluación de modelos de mayor escala, tanto open-source como comerciales, sobre circuitos de mayor complejidad semántica que la ALU, para determinar si el límite de flags aritméticos es superable con suficiente especialización de dominio.

A Fragmentos de Codigo RTL Representativos

A.1 Error E-ARI-01: SUB Incorrecto en Qwen2.5-Coder-7B V1

El siguiente fragmento ilustra el patron de error E-ARI-01: la operacion de resta utiliza una suma precalculada `(a+b)` y le resta 1, en lugar de computar directamente `a-b`. El resultado es funcionalmente incorrecto para todos los vectores de prueba de SUB.

```
// Implementacion INCORRECTA generada por Qwen2.5 V1
logic [8:0] sum;
assign sum = {1'b0, a} + {1'b0, b}; // solo suma ADD
assign temp_result = sum[7:0];

4'h1: begin  // SUB --- usa temp_result (=suma) y resta 1
    result = temp_result - 1;  // INCORRECTO: deberia ser a - b
    carry  = sum[8];           // carry de ADD, no borrow de SUB
end

// Implementacion CORRECTA
logic [8:0] temp_add, temp_sub;
assign temp_add = {1'b0, a} + {1'b0, b};  // ADD
assign temp_sub = {1'b0, a} - {1'b0, b};  // SUB
```

Listing 1: Qwen2.5-Coder-7B V1 — Error E-ARI-01: SUB usa suma precalculada.

A.2 Error E-FLAG-02: Overflow = Carry en Qwen2.5-Coder-7B V2

El error E-FLAG-02 es semanticamente mas sutil que E-ARI-01: el modelo captura correctamente el carry de la suma pero asigna ese mismo valor al flag de overflow,

ignorando que overflow en complemento a dos requiere comparar los bits de signo de los operandos con el del resultado.

```
4'b0000: begin  // ADD
    {carry, result} = a + b;  // carry capturado correctamente
    overflow = carry;          // INCORRECTO: overflow != carry
    // Overflow correcto en complemento a dos:
    // overflow = (~a[7] & ~b[7] & result[7]) |
    //            ( a[7] &  b[7] & ~result[7]);
end
```

Listing 2: Qwen2.5-Coder-7B V2 — Error E-FLAG-02: overflow definido como carry.

A.3 Error E-CAST-01: typedef enum sin Cast Explicito — Claude seq_det V1

El siguiente fragmento muestra el patron de codigo SystemVerilog generado por Claude (y de manera identica por ChatGPT y Gemini) que produce el error de compilacion en `iverilog -g2012`. El error aparece en la linea de asignacion del estado siguiente dentro del bloque `always_ff`.

```
// Generado por Claude / ChatGPT / Gemini --- todas las
   variantes V1-V5
typedef enum logic [1:0] {
    S0 = 2'b00,
    S1 = 2'b01,
    S2 = 2'b10,
    S3 = 2'b11
} state_t;

state_t state, next_state;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        state <= S0;     // Error linea ~29: cast explicito
           requerido
    else
        state <= next_state; // Error linea ~30
end

// Error iverilog -g2012:
```

```
// "This assignment requires an explicit cast."
// Correccion para iverilog:
//   state <= state_t'(S0);
//   state <= state_t'(next_state);
// Nota: el codigo es valido en ModelSim, VCS y Xcelium sin
   modificacion.
```

Listing 3: Claude seq_det V1 — Error E-CAST-01 (identico en ChatGPT y Gemini). Compilacion falla en iverilog: “This assignment requires an explicit cast”.

A.4 Error E-SEM-01: Latch Inferido — ChatGPT alu V2

ChatGPT en la variante V2 de la ALU genera un bloque `always_comb` donde la senal `tmp` no es asignada en todas las ramas del `case`, lo que Yosys interpreta como latch implicito y rechaza durante la sintesis (L4=FAIL, 0 celdas).

```
// Fragmento generado por ChatGPT V2 --- ALU
logic [8:0] tmp;

always_comb begin
    case (op)
        3'b000: tmp = {1'b0,a} + {1'b0,b};  // ADD
        3'b001: tmp = {1'b0,a} - {1'b0,b};  // SUB
        // ... otras operaciones ...
        // FALTA: default: tmp = 9'b0;
        // Sin default, tmp retiene su valor anterior en
           hardware
        // => Yosys infiere latch implicito => L4 FAIL
    endcase
end

// Correccion:
always_comb begin
    tmp = 9'b0;  // valor por defecto obligatorio en
       combinacional
    case (op)
        // ... mismo contenido ...
    endcase
end
```

Listing 4: ChatGPT alu V2 — Error E-SEM-01: latch inferido en sintesis. Mensaje Yosys: “Latch inferred for signal \alu.\tmp”.

B Datos Crudos del Benchmark — Modelos Comerciales

Table 12: Resultados completos (valores canonicos) — modelos comerciales. Promediados sobre 2 ejecuciones por variante.

Modelo	**Circuito**	**L1**	**L2 Func.**	**L3**	**L4 Celdas**	**Score**
Claude	adder_tree	✓	✓	×	137	75
Claude	alu (v1/2/3/5)	✓	×	×	344	40
Claude	alu (v4)	✓	×	×	361	40
Claude	seq_det	×	×	SKIP	0	0
ChatGPT	adder_tree	✓	✓	×	137	75
ChatGPT	alu (v1)	✓	✓	×	395	75
ChatGPT	alu (v2)	✓	✓	×	0 (latch)	60
ChatGPT	alu (v3)	✓	✓	×	399	75
ChatGPT	alu (v4)	✓	✓	×	424	75
ChatGPT	alu (v5)	✓	✓	×	395	75
ChatGPT	seq_det	×	×	SKIP	0	0
Gemini	adder_tree	✓	✓	×	137	75
Gemini	alu (v1/2/3/5)	✓	✓	×	395	75
Gemini	alu (v4)	✓	✓	×	400	75
Gemini	seq_det	×	×	SKIP	0	0

References

[1] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating Large Language Models for Verilog Code Generation," *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023.

[2] Y. Lu, S. Zheng, Q. Guo, and C. Zhang, "RTLBench: A Comprehensive Benchmark for RTL Code Generation with LLMs," *Proc. Design Automation Conf. (DAC)*, 2024.

[3] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.

[4] C. Wolf, "SymbiYosys: Front-end driver for Yosys-based formal hardware verification," GitHub Repository, 2023. [En linea]. Disponible: https://github.com/YosysHQ/sby

[5] C. Wolf, "Yosys Open Synthesis Suite," GitHub Repository, 2024. [En linea]. Disponible: https://github.com/YosysHQ/yosys

[6] S. Williams, "Icarus Verilog," GitHub Repository, 2024. [En linea]. Disponible: http://iverilog.icarus.com

[7] Z. Liu, C. Wu, Y. Liu, et al., "RTLCoder: Outperforming GPT-3.5 in Design RTL Generation with Our Open-Source Dataset and Lightweight Solution," *arXiv preprint arXiv:2312.08617*, 2024.

[8] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[9] D. Patterson and J. Hennessy, *Computer Organization and Design: ARM Edition*, 5th ed. Morgan Kaufmann, 2016.

[10] Qwen Team, Alibaba Cloud, "Qwen2.5-Coder Technical Report," *arXiv preprint arXiv:2409.12186*, 2024.

[11] A. Lozhkov et al., "StarCoder 2 and The Stack v2: The Next Generation," *arXiv preprint arXiv:2402.19173*, 2024.